

Práctica 4: Automatización y Generación de Código con IA en Canarias

Pipeline Dagster con IA generativa, mapas y sensores de eventos

Laura Lasso García

Marzo 2026

Índice general

1. Práctica 4: Automatización y Generación de Código con IA	2
1.1. Configuración de la rama de trabajo	2
1.2. Prueba del servicio de IA	2
1.3. Utilización lab test_prompt.py	3
1.4. Generación automática de código con IA y checks de calidad	4
1.5. Despliegue en GitHub Pages	5
1.6. Mapa de indicadores laborales por municipio	6
1.6.1. Mapa con ggplot (Dagster)	6
1.6.2. Mapa con Power BI (mapa personalizado)	6
1.7. Sensor de eventos: automatización del pipeline	8

Capítulo 1

Práctica 4: Automatización y Generación de Código con IA

1.1. Configuración de la rama de trabajo

Siguiendo las instrucciones de la práctica, se parte de la rama principal actualizada y se crea una nueva rama específica:

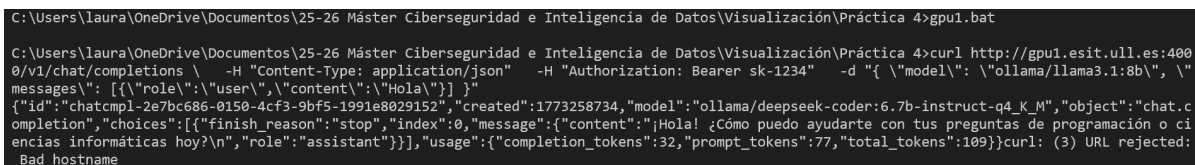
```
git checkout main
git pull
git checkout -b practica-calidad-checks-IA
```

Una vez finalizados todos los cambios, se realiza el commit y push correspondientes:

```
git add .
git commit -m "Practica 4: generacion de codigo IA, mapas y
sensor de eventos"
git push origin practica-calidad-checks-IA
```

1.2. Prueba del servicio de IA

Antes de integrar la IA en el pipeline, se probó la conexión al servicio mediante los ficheros .bat de la práctica. El fichero `gpu1.bat` verifica la conectividad con el endpoint. Tal como se puede observar en la imagen 1.1, funciona correctamente



```
C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>gpu1.bat
C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>curl http://gpu1.esit.ull.es:4000/v1/chat/completions \
-H "Content-Type: application/json" -H "Authorization: Bearer sk-1234" -d '{"model": "ollama/llama3.1:8b", "messages": [{"role": "user", "content": "¡Hola!"}], "stream": false}'
{"id": "chatcmpl-2e7bc686-0150-4cf3-9bf5-1991e8029152", "created": 1773258734, "model": "ollama/deepseek-coder:6.7b-instruct-q4_K_M", "object": "chat.completion", "choices": [{"finish_reason": "stop", "index": 0, "message": {"content": "¡Hola! ¿Cómo puedo ayudarte con tus preguntas de programación o ciencias informáticas hoy?\n", "role": "assistant"}}], "usage": {"completion_tokens": 32, "prompt_tokens": 77, "total_tokens": 109}}curl: (3) URL rejected: Bad hostname
```

Figura 1.1: Resultado ejecución `gpu.bat`.

El fichero `pyexpert.bat` pide al LLM que genere código Python para un bucle `for`, verificando que el modelo responde código ejecutable, que como se observa en la imagen 1.2, funciona correctamente. El fichero `plotnine.bat` pide al LLM que genere la función `generar_plot(df)` con punto focal aplicando el principio de Gestalt. Como se observa en la imagen 1.3.

```
C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>pyexpert.bat

C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>curl http://gpu1.esit.ull.es:4000/v1/chat/completions -H "Content-Type: application/json" -H "Authorization: Bearer sk-1234" -d "{ \"model\": \"ollama/llama3.1:8b\", \"messages\": [{\"role\": \"system\", \"content\": \"Eres un experto en Python. Vas a crear solo código python.\"}], {\"role\": \"user\", \"content\": \"Como hacer un bucle for\"}], \"temperature\": 0.7, \"max_tokens\": 500, \"top_p\": 0.9}"
{"id": "chatcmpl-26526a9d-2a49-4b14-85ea-1bd1aae0dfe6", "created": 1773258768, "model": "ollama/deepseek-coder:6.7b-instruct-q4_K_M", "object": "chat.completion", "choices": [{"finish_reason": "stop", "index": 0, "message": {"content": "Lo siento, pero tu pregunta no es clara. ¿Te estás preguntando cómo funciona un bucle 'for' en Python? Aquí está la sintaxis básica de un bucle 'for':\n\npython\nfor variable in iterable:\n    # código a ejecutar para cada iteración\n\nPor ejemplo, aquí es una instancia simple del uso de un for loop:\n\npython\nfruits = ['apple', 'banana', 'cherry']\nfor fruit in fruits:\n    print(fruit)\n\nEste programa imprimirá en pantalla cada fruta de la lista 'fruits', una por línea. ¿Esto es lo que buscabas? O tienes alguna otra pregunta relacionada con Python o programación en general?\n"}}, {"role": "assistant"}], "usage": {"completion_tokens": 174, "prompt_tokens": 103, "total_tokens": 277}}
```

Figura 1.2: Resultado ejecución pyexpert.bat.

```
C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>plotnine.bat

C:\Users\laura\OneDrive\Documentos\25-26 Máster Ciberseguridad e Inteligencia de Datos\Visualización\Práctica 4>curl http://gpu1.esit.ull.es:4000/v1/chat/completions -H "Content-Type: application/json" -H "Authorization: Bearer sk-1234" -d "{ \"model\": \"ollama/llama3.1:8b\", \"messages\": [{\"role\": \"system\", \"content\": \"Eres un experto en Python. Vas a crear solo código python. Además eres un experto en Gestalt. No des explicaciones.\"}], {\"role\": \"user\", \"content\": \"Tengo un DataFrame con columnas [año, isla, medida, valor]. Islas: [Tenerife, Gran Canaria]. Genera código Python con plotnine: la función debe llamarse generar_plot(df), destacar Tenerife en azul (#007bff) y el resto en gris (#D3D3D3) para aplicar Punto Focal.\"}], \"temperature\": 0.7, \"max_tokens\": 500, \"top_p\": 0.9}"
{"id": "chatcmpl-ab016948-db1a-4b8c-b3d4-ecb3a76fe8e2", "created": 1773258777, "model": "ollama/deepseek-coder:6.7b-instruct-q4_K_M", "object": "chat.completion", "choices": [{"finish_reason": "stop", "index": 0, "message": {"content": "\npython\nimport plotnine as gg\nfrom plotnine import aes, geom_point, facet_wrap\ntheme\n\ndef generar_plot(df):\n    df['color'] = ['#007bff' if isla == 'Tenerife' else '#D3D3D3' for isla in df['isla']]\n    plot = (gg.ggplot(df, aes('medida', 'valor')) +\n            geom_point(aes(color='factor(isla)'), shape='.') +\n            facet_wrap('~ año', ncol = 2) +\n            theme(figure_size = (10,8), dpi = 100))\n    return plot\n\n"}}, {"role": "assistant"}], "usage": {"completion_tokens": 178, "prompt_tokens": 222, "total_tokens": 400}}
```

Figura 1.3: Resultado ejecución plotnine.bat.

1.3. Utilización lab test_prompt.py

Para la primera ejecución del archivo, se debe ejecutar la siguiente orden:

```
python -m dagster dev -f test-prompt.py
```

Una vez ejecutado, el código dará como resultado el gráfico de la imagen 1.4.

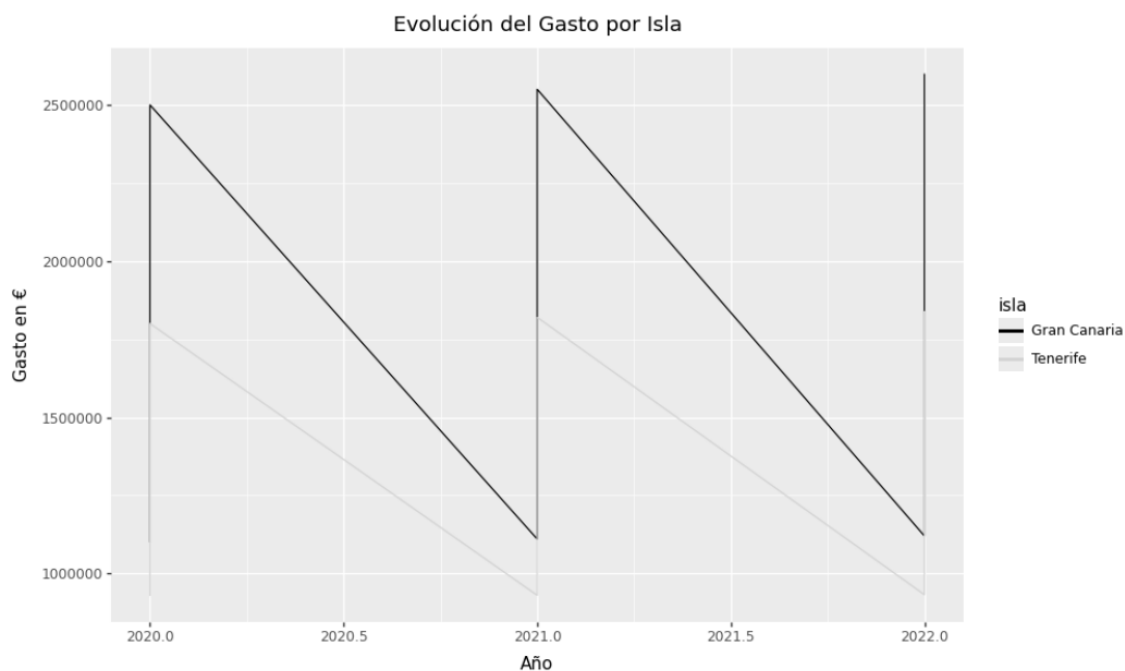


Figura 1.4: Resultado de la ejecución del código de test_prompt.py.

Luego, para los checks necesarios, se han incluido los de la siguiente tabla. Más adelante, se especificarán los checks que se han incluido en el proyecto de rentas.

Check	Asset	Qué valida
check_estandarizacion_islas	islas_raw	Nombres de islas consistentes (Gestalt: Similitud)
check_columnas_requeridas	islas_raw	Que existan las columnas año, isla, medida, valor
check_sin_nulos	islas_raw	Sin valores nulos en columnas clave
check_codigo_contiene_funcion	codigo_generado_ia	Que la IA generó <code>def generar_plot(df)</code>
check_focal_clarity	visualizacion_png	Confirmación del Punto Focal aplicado

Cuadro 1.1: Checks de calidad implementados en `test_checks.py`

1.4. Generación automática de código con IA y checks de calidad

Las tres plantillas (`template_ia_canarias`, `template_ia_islas`, `template_ia_estudios`) comparten la misma arquitectura de prompt y están diseñadas en torno a tres decisiones concretas de ingeniería.

Separación system/user y fijación de librería. El mensaje de sistema declara explícitamente el rol del modelo como experto en gramática de gráficos y Plotnine, e indica que debe devolver *exclusivamente código Python*. Esto evita dos fallos frecuentes en modelos pequeños como llama3.1:8b: usar `matplotlib` en lugar de Plotnine, e incluir texto explicativo mezclado con el código que rompe el `exec()` posterior.

Contrato de interfaz fijo mediante template. Los tres prompts inyectan siempre la misma firma `def generar_plot(df): ... return plot`. Este contrato garantiza que el código generado sea invocable de forma determinista con `entorno['generar_plot'](df)` sin depender del nombre que el modelo elija libremente. El check `check_codigo_generado_*` valida precisamente que este contrato se cumple antes de ejecutar el código.

Restricciones negativas basadas en fallos observados. Durante el desarrollo se detectaron tres patrones erróneos recurrentes que se corrigieron añadiendo advertencias explícitas en mayúsculas al prompt:

- `figure_size` colocado dentro de `theme_minimal()` en lugar de `theme()` → “*IMPORTANTE: figure_size va SIEMPRE dentro de theme()*”
- Omisión de `type='qual'` en `scale_fill_brewer` (pipeline de estudios) → “*type='qual' es OBLIGATORIO*”
- Uso incorrecto de `scale_color_manual` sin respetar la paleta inyectada (pipeline de islas) → los colores se pasan como valor literal en el prompt y se sobrescriben además en el asset de renderizado

Mecanismo de fallback y checks de salida. Dado que el modelo puede generar código sintácticamente válido pero semánticamente incorrecto, cada asset de renderizado

implementa dos niveles de recuperación: primero reintenta con el código IA añadiendo `scale_color_manual` forzado, y si el `.save()` falla reconstruye el gráfico desde código determinista (`grafico_limpio()`). Los checks de salida `check_grafico_*` verifican independientemente que el PNG resultante existe y supera los 10 KB.

Checks de calidad por pipeline

Asset	Check	Qué verifica
raw_renta_canarias	check_raw_renta_canarias	Columnas obligatorias + sin en OBS_VALUE
cleaned_canarias	check_cleaned_canarias	Registros con código ES70 > 0
transformed_canarias	check_transformed_canarias	Las 5 medidas presentes tr replace
viz_data_canarias	check_viz_data_canarias	Sin nulos + al menos 5 años dis
codigo_generado_canarias	check_codigo_generado_canarias	Contiene generar_plot, ggpl return
grafico_evolucion_canarias	check_grafico_evolucion_canarias	PNG existe y > 10 KB
raw_codislas	check_raw_codislas	Columnas CPRO/CMUN/IS NOMBRE + registros > 0
cleaned_municipios	check_cleaned_municipios	Al menos 10 municipios con c de 5 dígitos
municipios_con_islas	check_municipios_con_islas	< 10 % de nulos en ISLA tras e
top_5_municipios	check_top_5_municipios	Exactamente 5 municipios dist
codigo_generado_islas	check_codigo_generado_islas	Contiene generar_ facet_wrap y return
grafico_top5_evolucion_medidas	check_grafico_top5	PNG existe y > 10 KB
raw_nivel_estudios	check_raw_nivel_estudios	Columnas del Excel + registro
cleaned_nivel_estudios	check_cleaned_nivel_estudios	Los 5 niveles educativos espe presentes
educacion_temporal_por_isla	check_educacion_temporal	Datos de 2023 presentes + po tajes suman ~100 %
codigo_generado_estudios	check_codigo_generado_estudios	Contiene generar_plot, geor y return
viz_educacion_isla_2023	check_viz_educacion_isla_2023	PNG existe y > 10 KB

Cuadro 1.2: Checks de calidad implementados con `@asset_check` en los tres pipelines

1.5. Despliegue en GitHub Pages

Se vinculó el repositorio a la rama `gh-pages` para publicar automáticamente las imágenes generadas. Los pasos para crear la rama fueron:

```
git checkout --orphan gh-pages
git rm -rf .
echo "GitHub Pages" > README.md
git add README.md
git commit -m "Init gh-pages"
git push origin gh-pages
git checkout practica-calidad-checks-IA
```

Se añadió la función `subir_imagen_a_ghpages()` en cada `.py` de `assets` para subir automáticamente las imágenes generadas en cada ejecución del pipeline. Una vez activado GitHub Pages en *Settings* → *Pages* → *Source* → *gh-pages*, las imágenes quedaron disponibles públicamente en las siguientes URLs:

- https://lauralasso.github.io/dagster-canarias-analytics/grafico_evolucion_canarias.png
- https://lauralasso.github.io/dagster-canarias-analytics/grafico_top5_evolucion_medidas.png
- https://lauralasso.github.io/dagster-canarias-analytics/perfil_educativo_isla_2023.png
- https://lauralasso.github.io/dagster-canarias-analytics/mapa_renta_municipios.png

1.6. Mapa de indicadores laborales por municipio

1.6.1. Mapa con ggplot (Dagster)

Se creó el archivo `mapa_canarias_ia.py` en la carpeta `assets`. Antes de integrarlo en producción, se desarrolló un script de laboratorio (`test_mapa.py`) para verificar que el gráfico se generaba correctamente. El proceso técnico fue:

```
pip install geopandas
```

El asset carga el fichero GeoJSON del ISTAC (`Municipios-2024.json`), que contiene las coordenadas de los polígonos de los 88 municipios de Canarias en `features` → `geometry` y los indicadores laborales de la EPA-Reg en `features` → `properties`. Las columnas reales del fichero son `label` (nombre del municipio) y los indicadores `tpar_t` (tasa de paro total), `tsal_t` (tasa de salarización), entre otros.

La lógica del asset detecta automáticamente las columnas disponibles para evitar errores por nombres inexistentes:

```
if COL_VALOR not in gdf.columns:
    numericas = gdf.select_dtypes(include='number').columns.
        tolist()
    COL_VALOR = numericas[0] if numericas else None
```

1.6.2. Mapa con Power BI (mapa personalizado)

Para crear el mapa en Power BI se siguieron los siguientes pasos:

1. **Conversión a TopoJSON:** Se descargó el fichero `Municipios-2024.json` y se transformó a TopoJSON mediante <https://mapshaper.org/>, exportándolo como `municipios-canarias.json`.
2. **Generación del CSV con los datos:** Se ejecutó un script Python que extrae las propiedades del GeoJSON y las guarda como tabla plana `municipios_indicadores.csv`:

```
gdf = gpd.read_file("Municipios-2024.json")
cols = ['label', 'tpar_t', 'tpar_m', 'tpar_f', 'tsal_t', 'tsal_m', 'tsal_f', ...]
gdf[cols].to_csv("municipios_indicadores.csv", index=False)
```

3. **Activación del visual Shape Map:** En Power BI se activó la característica en versión preliminar:
Archivo → Opciones → Características en versión preliminar → Activar mapa de formas → Reiniciar Power BI
4. **Carga de datos:** Se importó `municipios_indicadores.csv` mediante *Inicio → Obtener datos → CSV*.
5. **Configuración del mapa:** Se insertó el visual **Mapa de formas** y se configuró:
Formato → Configuración del mapa → Tipo de mapa → Mapa personalizado → Agregar un tipo de mapa → municipios-canarias.json
6. **Asignación de campos:**
 - **Ubicación:** `label`
 - **Saturación de color:** `tpar_t` (tasa de paro total)
 - **Información sobre herramientas:** `tsal_t`, `tpar_m`, `tpar_f`
7. **Ajuste de colores:**
 - Color mínimo: `#d4e6f1` (azul claro)
 - Color máximo: `#1a5276` (azul oscuro)

El resultado, reflejado en la imagen 1.5, es un mapa coroplético de los 88 municipios de Canarias coloreados según la tasa de paro, con la misma paleta que el mapa generado con ggplot para mantener coherencia visual.

Mapa de rentas en Canarias

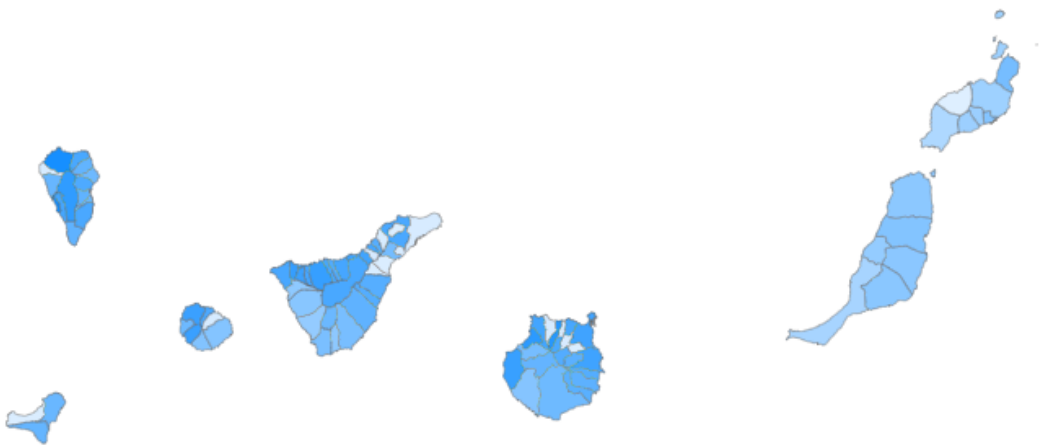


Figura 1.5: Mapa de rentas en Canarias en Power BI.

1.7. Sensor de eventos: automatización del pipeline

Para lanzar automáticamente todo el proceso de Dagster cuando se detecten cambios en la carpeta de datos del proyecto, se implementó un **sensor nativo de Dagster** en el fichero `definitions.py`. Esta solución es preferible al uso de `watchdog` externo porque todo queda integrado en la UI de Dagster, que gestiona el estado, los reintentos y el historial de ejecuciones.

El sensor utiliza un hash MD5 de la carpeta `/data` (combinando nombres de fichero y fechas de modificación) como cursor. Cada 30 segundos compara el hash actual con el almacenado; si difieren, lanza el job `todo_el_pipeline`:

```
import hashlib
from pathlib import Path
from dagster import (
    Definitions, load_assets_from_modules,
    load_asset_checks_from_modules,
    sensor, RunRequest, DefaultSensorStatus, define_asset_job,
    AssetSelection,
)

from src.dagster_app.assets import (
    renta_canarias_ia, renta_islas_ia, nivel_estudios_ia,
    mapa_canarias_ia
)

CARPETA_DATOS = Path(__file__).resolve().parent.parent.parent / "
    data"

all_assets = load_assets_from_modules(
    [renta_canarias_ia, renta_islas_ia, nivel_estudios_ia,
     mapa_canarias_ia]
)
all_checks = load_asset_checks_from_modules(
    [renta_canarias_ia, renta_islas_ia, nivel_estudios_ia]
)

todo_el_pipeline = define_asset_job(
    name="todo_el_pipeline",
    selection=AssetSelection.all()
)

def _hash_carpeta(path: Path) -> str:
    h = hashlib.md5()
    for f in sorted(path.rglob("*")):
        if f.is_file() and f.suffix not in {".pyc", ".tmp"}:
            h.update(str(f).encode())
            h.update(str(f.stat().st_mtime).encode())
    return h.hexdigest()

@sensor(
    job=todo_el_pipeline,
```

```

    default_status=DefaultSensorStatus.RUNNING,
    minimum_interval_seconds=30,
    description="Lanza el pipeline completo cuando cambia algun
        archivo en /data"
)
def sensor_cambio_datos(context):
    if not CARPETA_DATOS.exists():
        context.log.warning(f"Carpeta no encontrada: {
            CARPETA_DATOS}")
        return

    nuevo_hash = _hash_carpeta(CARPETA_DATOS)
    ultimo_hash = context.cursor or ""

    if nuevo_hash != ultimo_hash:
        context.update_cursor(nuevo_hash)
        context.log.info("Cambio detectado en /data - lanzando
            pipeline completo")
        yield RunRequest(run_key=nuevo_hash)
    else:
        context.log.info("Sin cambios en /data")

defs = Definitions(
    assets=all_assets,
    asset_checks=all_checks,
    jobs=[todo_el_pipeline],
    sensors=[sensor_cambio_datos],
)

```

El sensor aparece en *Automation* → *Sensors* de la UI de Dagster con estado **Running** activado automáticamente gracias a `DefaultSensorStatus`, tal y como se observa en la imagen 1.6. **RUNNING**, sin necesidad de activarlo manualmente. Cada vez que se añade o modifica un fichero en la carpeta `/data`, el pipeline completo se lanza en menos de 30 segundos.

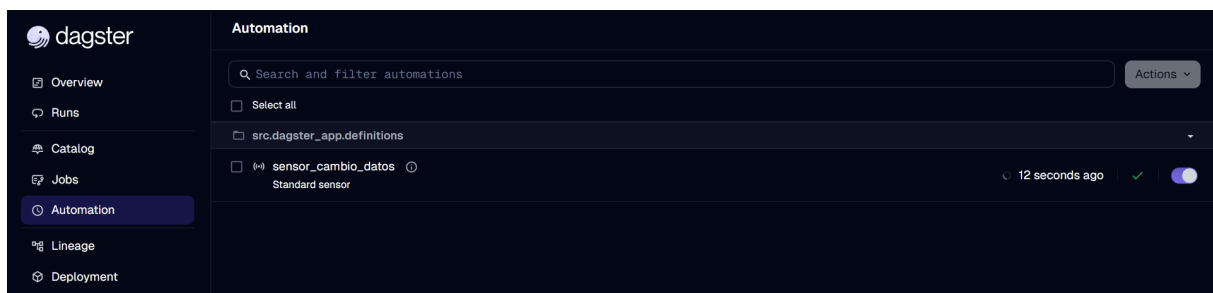


Figura 1.6: Estado del sensor programado, que está activo.

Una vez realizados los cambios, se guardaron en la rama de trabajo:

```

git add src/dagster_app/definitions.py
git add src/dagster_app/assets/mapa_canarias_ia.py
git commit -m "feat: sensor nativo de cambios en /data y mapa de
    municipios"

```

```
git push origin practica-calidad-checks-IA
```

Bibliografía

- [1] Lasso García, L. (2026). *Repositorio del Proyecto: Análisis de Renta y Educación en Canarias*. GitHub. <https://github.com/LauraLasso/dagster-canarias-analytics/tree/practica-calidad-checks-IA>
- [2] Instituto Canario de Estadística (ISTAC). *Indicadores laborales. Municipios de Canarias. 2024*. Recuperado de: <https://www3.gobiernodecanarias.org/istac>
- [3] Dagster Labs. (2026). *Dagster Documentation: Sensors and Automation*. <https://docs.dagster.io/concepts/partitions-schedules-sensors/sensors>
- [4] Hassan K. (2024). *Plotnine: A Grammar of Graphics for Python*. <https://plotnine.org/>
- [5] GeoPandas developers. (2024). *GeoPandas Documentation*. <https://geopandas.org/>
- [6] Wertheimer, M. (1923). *Laws of Organization in Perceptual Forms (Gestalt Theory)*. Explicación aplicada a la visualización de datos.
- [7] Eremin, N., & Otros. (2023). *DataOps Principles: Continuous Integration and Quality Control in Data Pipelines*.